



Blazor e AI: UI reattive tra streaming e agenti



Andrea Dottor

Senior Developer & Consultant
Microsoft MVP Developer Technologies



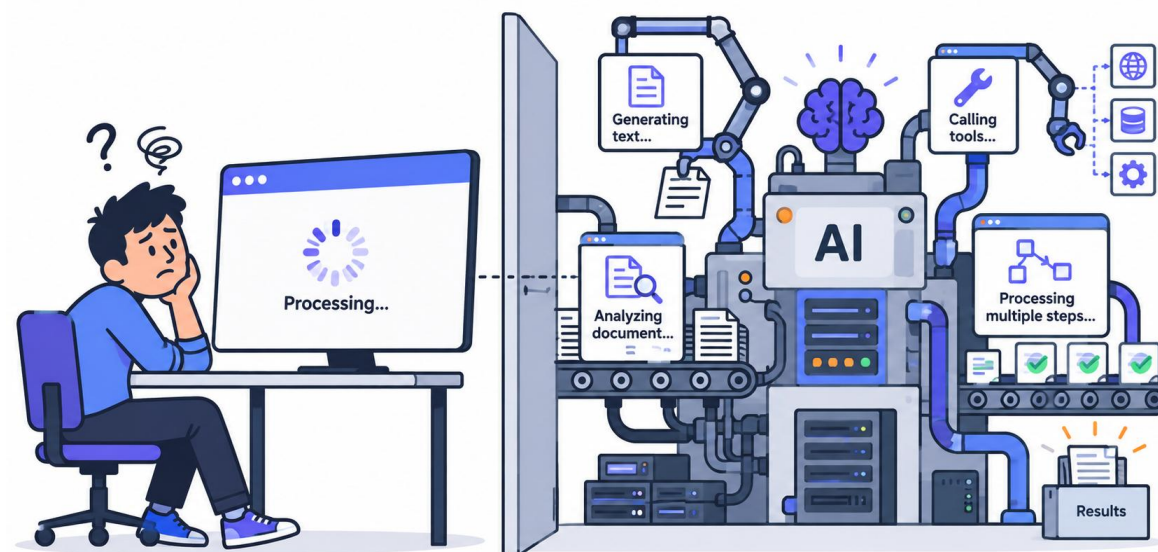


Quando l'AI non risponde subito

Siamo abituati ad applicazioni che rispondono velocemente.

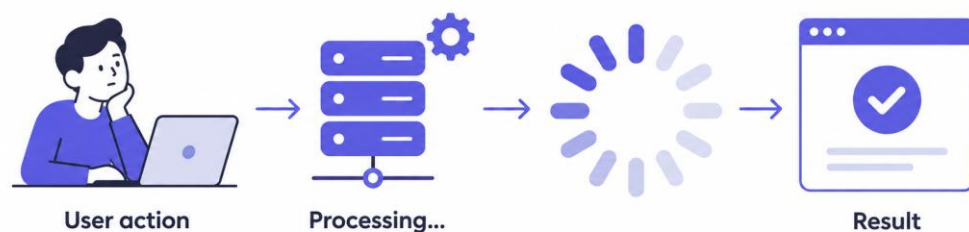
Con l'AI non è sempre così:

- generazione di testo
- analisi di documenti
- tool invocation
- pipeline con più step
- Agenti



**Aspettare qualche secondo non è necessariamente un problema.
Non sapere cosa sta succedendo, sì.**

Il classico spinner



L'applicazione sta lavorando.
Ma l'utente cosa sa?



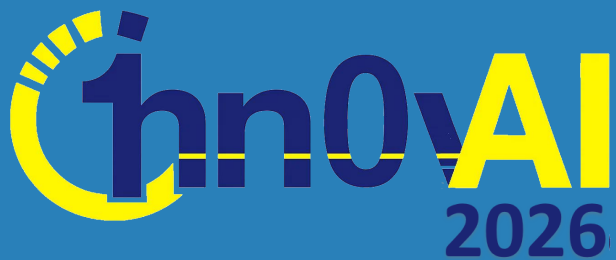
Task<T>: aspetto il risultato

```
Task<string> response = await service.AskAsync(prompt);
```

- semplice
- leggibile
- perfettamente corretto

Ma:

- Il risultato esiste solamente quando l'operazione è terminata.



DEMO 1:

Aspetto la risposta

E se il risultato arrivasse un po' alla volta?

streaming

IAsyncEnumerable<T>

Un risultato asincrono...

...che può produrre più valori nel tempo.

```
await foreach (var item in stream)
{
    ...
}
```

Non aspetto il completamento per iniziare a usare il risultato.

Task<T> vs IEnumerable<T>

Task<T>



One final result

IAsyncEnumerable<T>



Multiple results over time

Streaming con IChatClient

GetResponseAsync(...)



Task<ChatResponse>

GetStreamingResponseAsync(...)



IAsyncEnumerable<ChatResponseUpdate>

Microsoft.Extensions.AI espone proprio la risposta streaming di *IChatClient* come *IAsyncEnumerable<ChatResponseUpdate>*.

Blazor + streaming

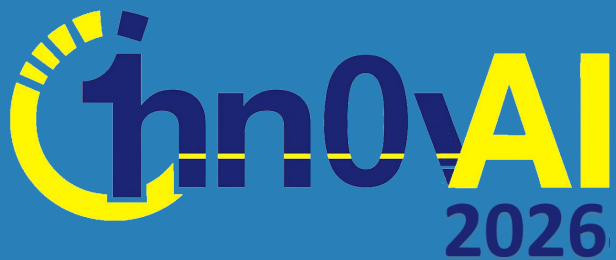
```
await foreach (var update in service.AskStreamingAsync(...))  
{  
    response += update;  
}
```

cambia lo stato



Blazor aggiorna la UI

Attenzione: utilizzare con testa lo StateHasChanged!



DEMO 2:

Streaming della risposta

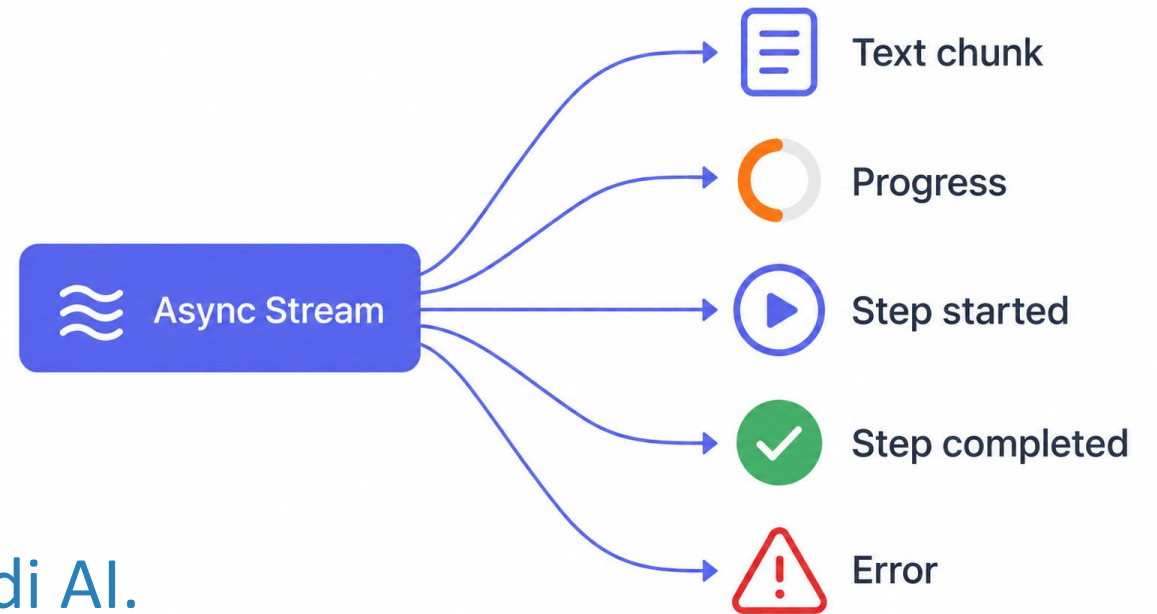


Streaming ≠ token

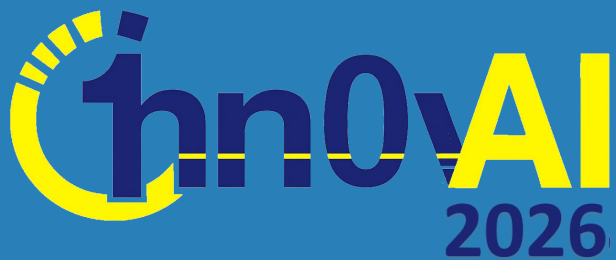
Quando pensiamo allo streaming AI pensiamo subito ai token.

Ma possiamo ricevere anche:

- avanzamento
- step iniziati
- step completati
- risultati intermedi
- errori



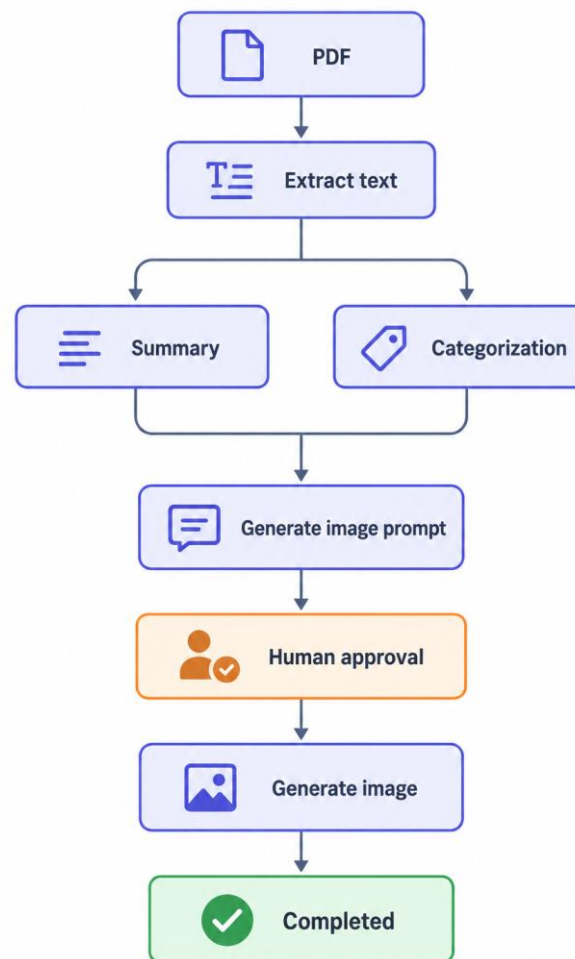
`IAsyncEnumerable<T>` non sa nulla di AI.
È solamente uno stream asincrono di dati.



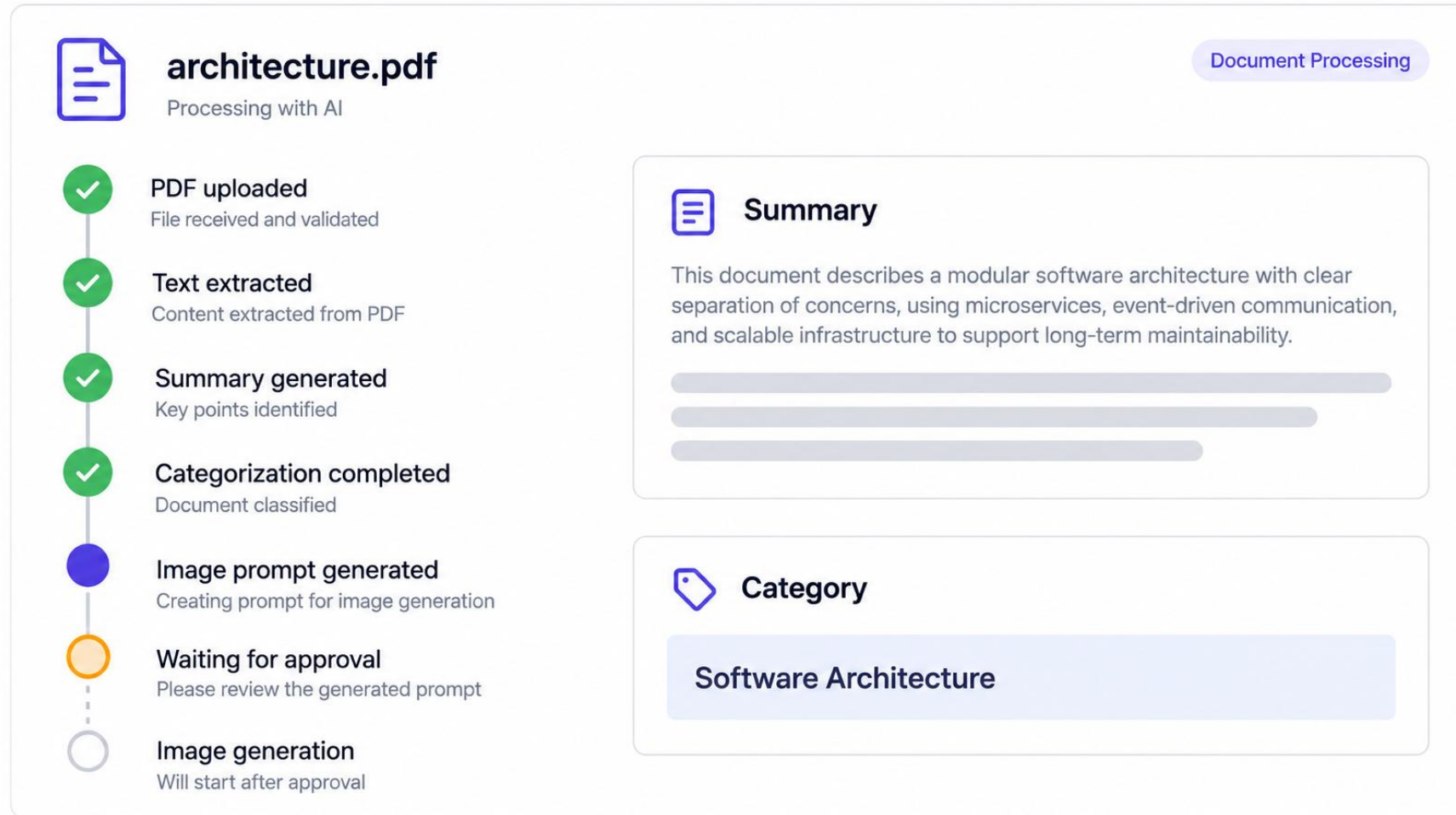
PIPELINE AI



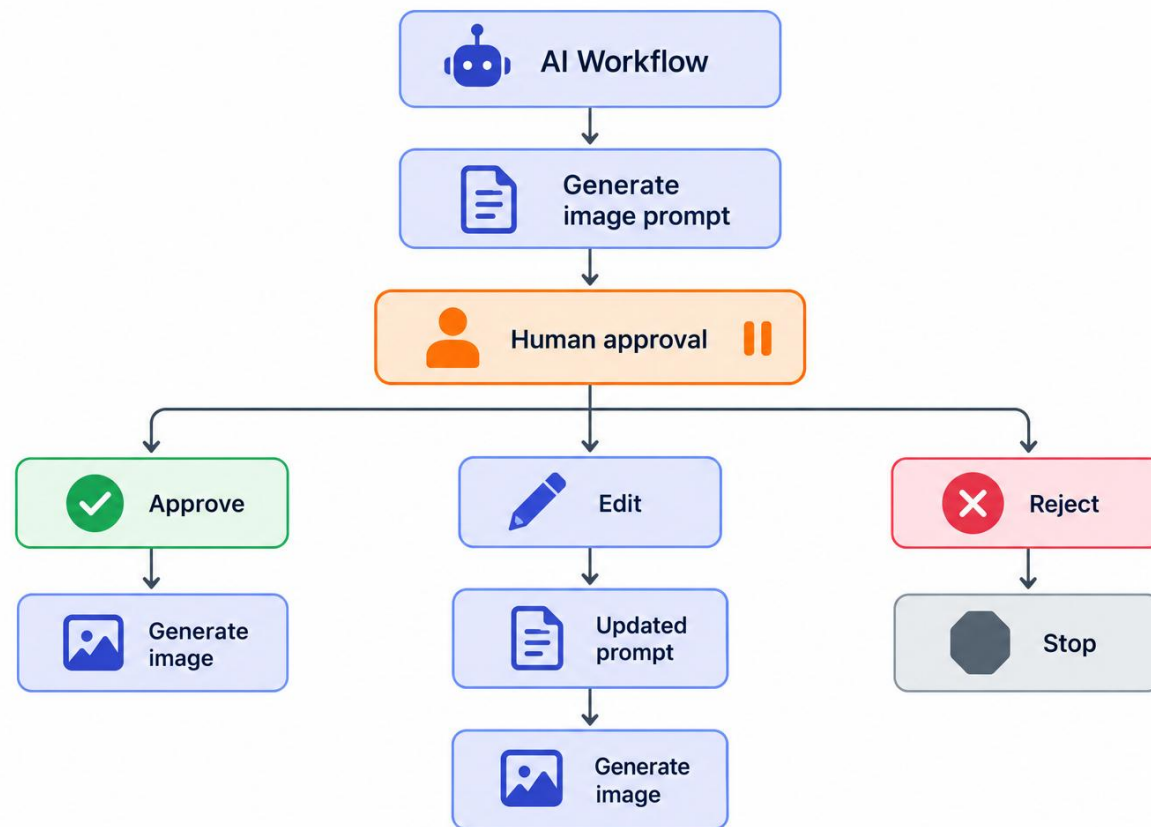
Un PDF, più attività



Cosa vorrei mostrare all'utente?



La UI non deve solamente osservare



La UI diventa parte del workflow

Microsoft Agent Framework

Un workflow produce eventi durante la propria esecuzione.

```
await foreach (WorkflowEvent evt in run.WatchStreamAsync())  
{  
    ...  
}
```

Tra gli eventi puoi osservare, ad esempio:

- executor avviato
- executor completato
- output
- errore

WatchStreamAsync() espone proprio un `IAsyncEnumerable<WorkflowEvent>`, quindi il modello si inserisce molto bene nel ragionamento fatto fino a qui.

Non portare il framework nella UI

WorkflowEvent/Request



map



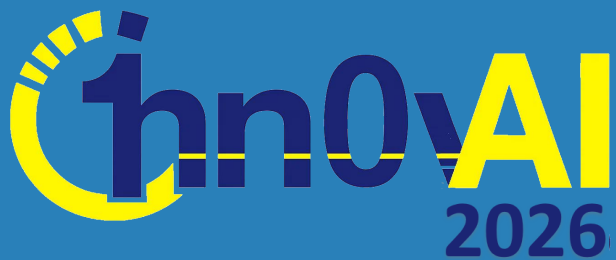
PipelineUpdate



Blazor

```
public record PipelineUpdate(  
    string Step,  
    PipelineStatus Status,  
    string? Message);
```

**La UI deve conoscere il processo,
non Agent Framework.**



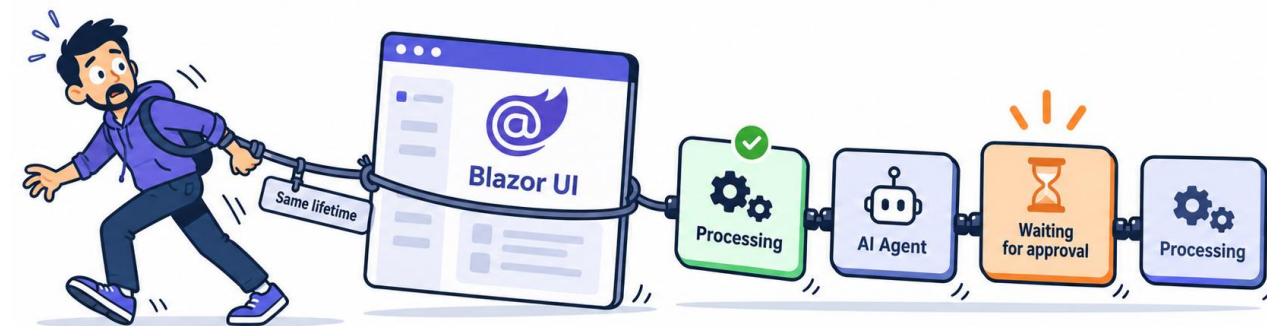
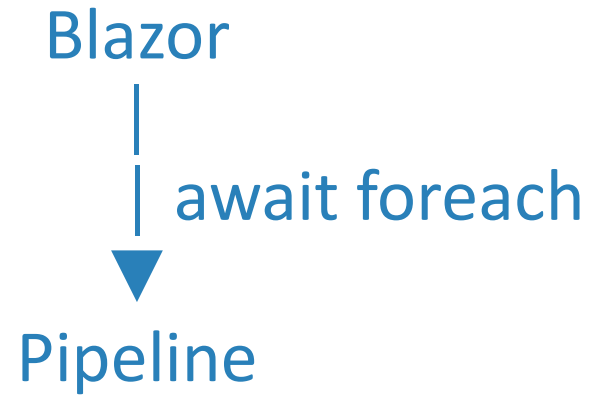
DEMO 3:

Dal token alla pipeline



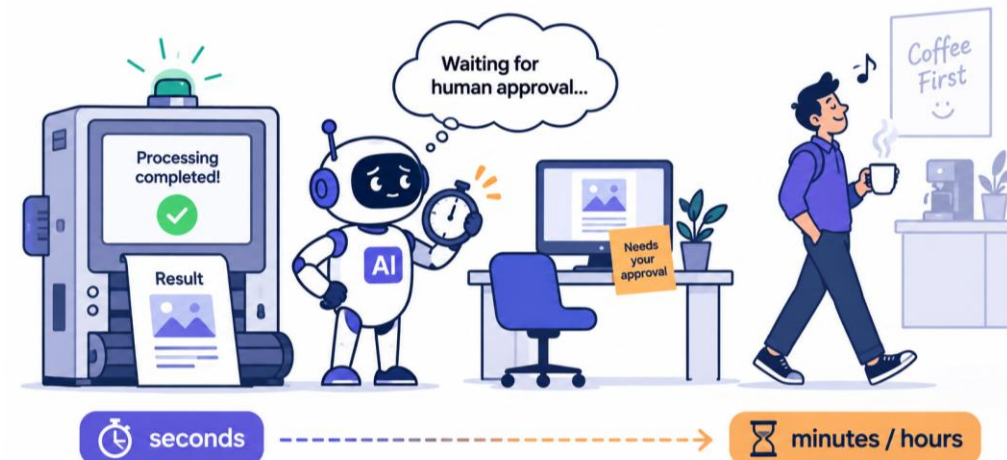
Abbiamo davvero risolto tutto?

La UI è ancora il consumer diretto del processo.



Un workflow può anche aspettare

- Un modello risponde in secondi.
Una persona potrebbe rispondere dopo minuti... o ore.
- Il lifetime del workflow inizia a essere diverso dal lifetime dell'interazione UI



Cambia il requisito

La pipeline ora può durare molto di più.
Il processo può rimanere in attesa

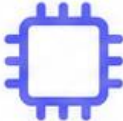
- l'elaborazione può durare pochi secondi
- l'approvazione può arrivare molto più tardi
- UI e processo iniziano ad avere lifetime differenti



È ancora corretto tenere tutto legato a un await foreach?

Channel<T>



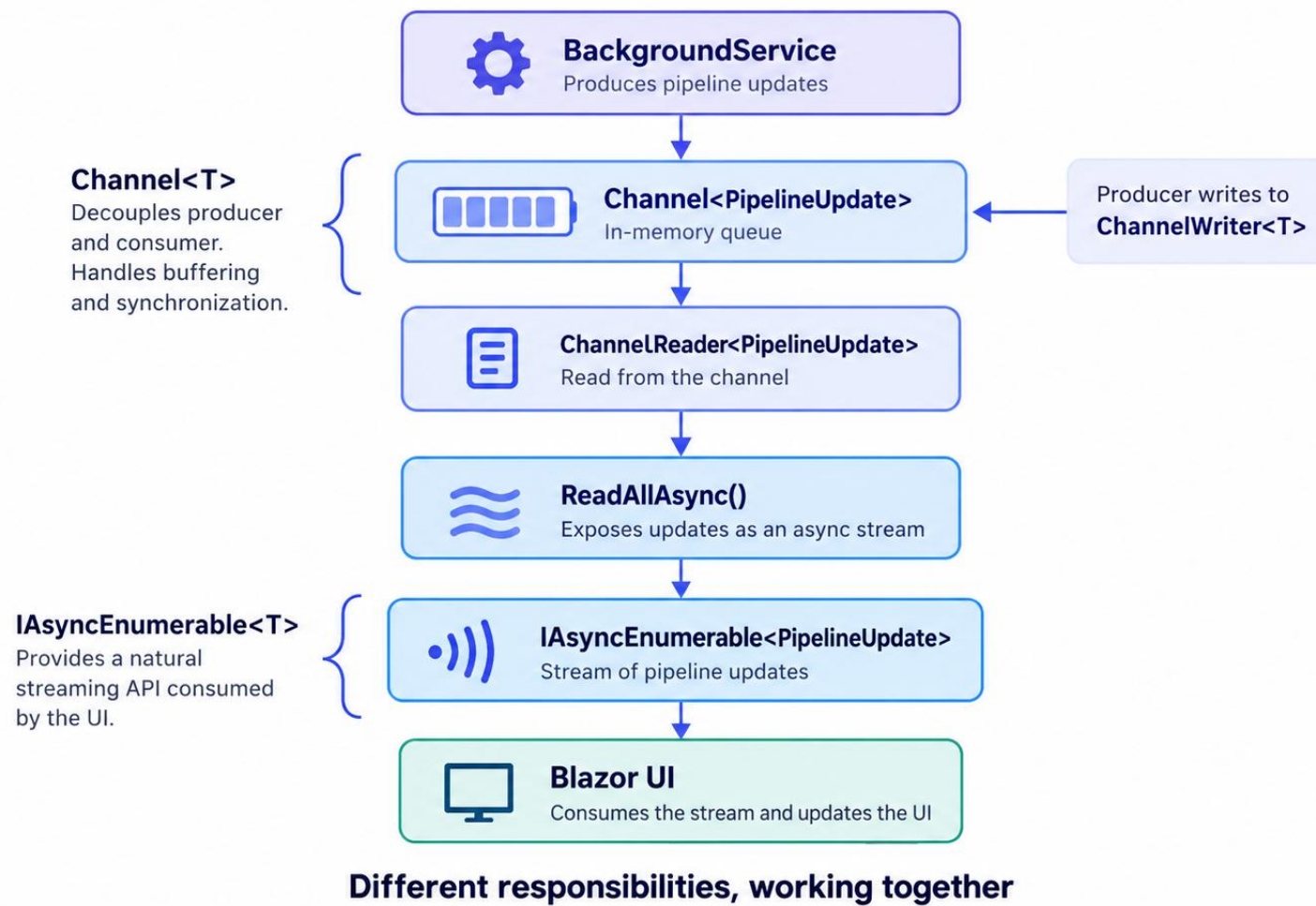

In memory


Asynchronous


Decoupled

Channel $\neq$ message broker

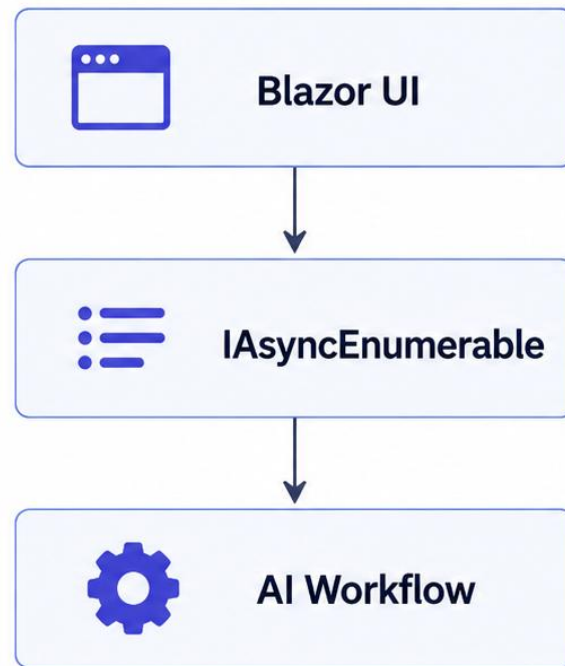
Channel e IAsyncEnumerable insieme



Non sono alternative.
Risolvono problemi
differenti.

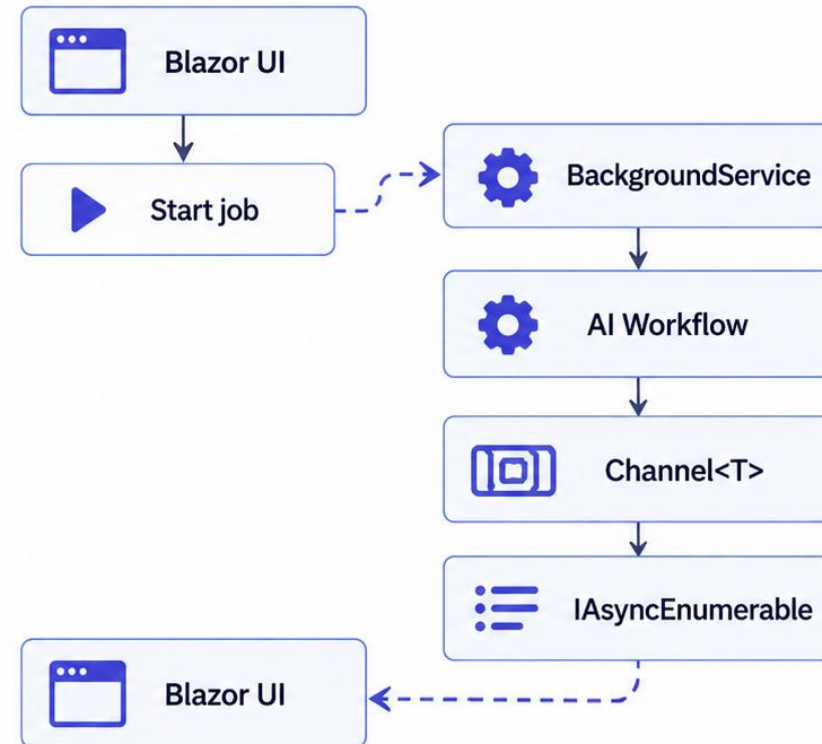
Demo 3 vs Demo 4

Direct execution



UI directly observes the workflow

Background execution



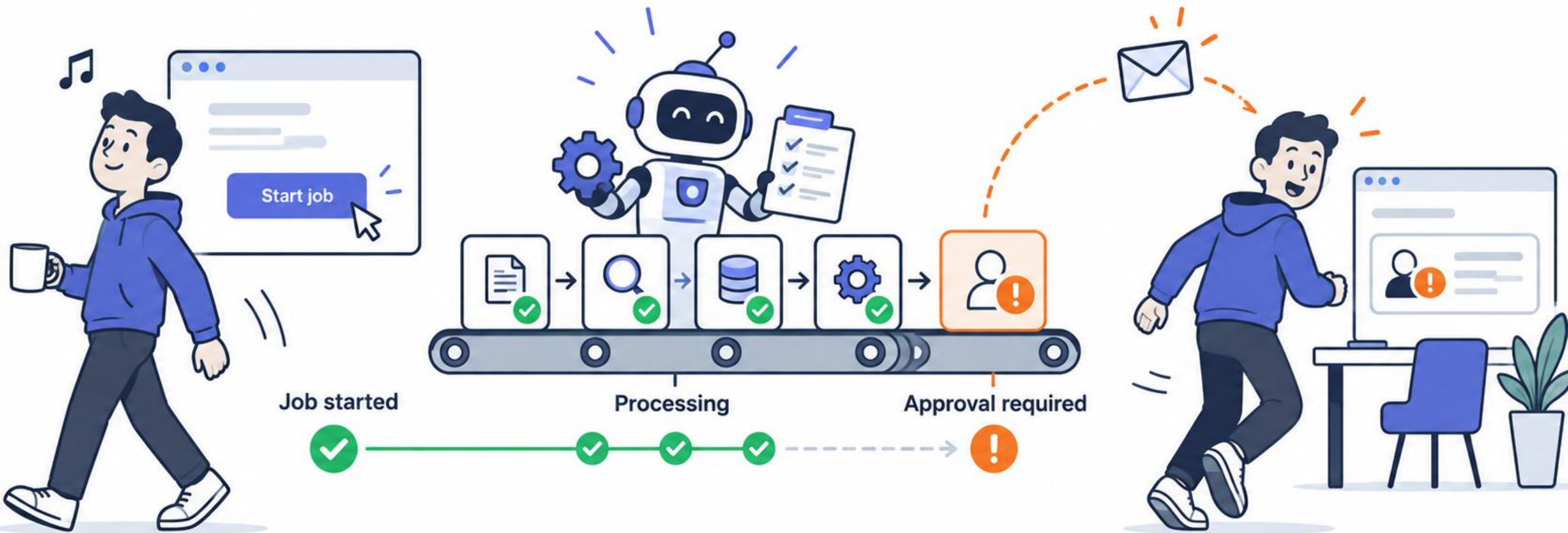
Workflow runs independently from the UI



DEMO 4:

Disaccoppiare UI e pipeline





Fin dove arriva Channel<T>?

Channel<T>:

- vive nel processo
- non è persistente
- non sopravvive al restart
- non risolve il multi-instance
- non è automaticamente pub/sub

Serve durabilità?

Serve attraversare processi?

Serve scalare su più istanze?



Message broker / persistent storage

Quale scegliere?

Esigenza	Strumento
Mi serve il risultato finale	Task<T>
Voglio risultati progressivi	IAsyncEnumerable<T>
Voglio disaccoppiare producer e consumer nello stesso processo	Channel<T>
Voglio durabilità / processi differenti	broker / storage

**Non esiste il migliore.
Dipende dal problema.**

4 cose da ricordare

- **async non significa background**

Un'operazione asincrona può essere ancora direttamente legata al chiamante.

- **Streaming non significa solamente token**

Può rappresentare qualsiasi aggiornamento progressivo.

- **IAsyncEnumerable<T> e Channel<T> non sono concorrenti**

Possono lavorare insieme.

- **La UI deve raccontare cosa sta succedendo**

Non lasciare l'utente davanti a uno spinner quando hai informazioni migliori.

Grazie!

```
{  
  name:      Andrea Dottor  
  email:     andrea@dottor.net  
  bsky:      @andrea.dottor.net  
  linkedin:  @andreadottor  
}
```

